INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, SRI CITY • CSE

Zero-Overhead Prefetch-Resilient Last-Level Cache Policy for Multicore Systems

B.Tech Project Mid-Evaluation Review • Academic Year 2026–2027

Candidate

Chirag Sharma

Roll No: **S20240010223**

B.Tech, Computer Science & Engineering

Project Advisor

Dr. Bheemappa Halavar

Assistant Professor

Dept. of Computer Science & Engineering

Cycle-Accurate ChampSim Simulation • SPEC CPU Benchmarks • 8MB Shared LLC

1. Motivation & Multicore Cache Contention

Hardware Prefetching Paradox

- **Regular Streams (`gcc`, `lbm`):** Spatial stride prefetchers achieve $> 50\%$ accuracy, effectively hiding memory latency.
- **Irregular Streams (`429.mcf`):** Pointer chasing causes accuracy to drop below $< 1\%$.
- **Aggressive Speculation:** Hardware prefetchers issue speculative requests without feedback on LLC utility.

Multicore Contention & Blindness

- **Prefetch-Oblivious Policies:** Standard LRU and SRRIP insert prefetched lines at high priority ($RRPV = 2$ / MRU).
- **Cross-Core Interference:** Inaccurate prefetches evict valid demand blocks of co-running compute cores.
- **Severe Throughput Loss:** Unchecked prefetch thrashing causes up to **50%+ IPC collapse** in shared multicore LLCs.

Research Goal: *Design an LLC policy that neutralizes prefetch pollution with **ZERO hardware storage overhead** (0 extra metadata bits).*

2. Literature Review & Identified Research Gaps

State-of-the-Art Approaches

- **PACMan (IEEE CAL 2023):**
  Prefetch-aware cache management via dual queues; requires dedicated accuracy counters and sampling sets.
- **DRRIP (ISCA 2010):**
  Dynamic RRIP using set-dueling PSEL counters; oblivious to prefetch vs. demand stream origin.
- **SHiP (ISCA 2011):**
  Signature-based hit predictor; requires an **8,192-entry SHCT table (2,048 Bytes SRAM)**.
- **Hawkeye (ISCA 2016):**
  Learns optimal cache replacement but requires extensive hardware history tables.

Identified Research Gaps

1. **High Storage Overhead:**
   Existing predictive policies require dedicated SRAM tables (2KB in SHiP) or complex sampling logic.
2. **Lack of Zero-Overhead Resilience:**
   Standard high-performance caches cannot afford extra SRAM tables solely for prefetch tracking.
3. **Our Proposed Solution:**
   A **Zero-Overhead** policy (`PA-SRRIP`) and a **0.75-Byte Self-Tuning** controller (`PA-Adaptive`) that eliminates prefetch pollution right at insertion.

Dual-Mechanism Architectural Design

1. Demoted Initial Insertion (0 Extra Bits):

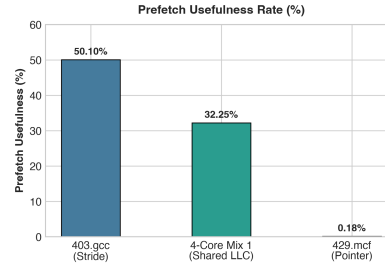
- Demand fills → Inserted at normal priority ($RRPV = 2$).
- Prefetch fills → Demoted to victim status ($RRPV = 3$).
- Inaccurate blocks are evicted first without polluting LLC.

2. Self-Adaptive Promotion on Demand Hit:

- On first demand hit, promoted to MRU ($RRPV = 0$).
- Line is reclassified as verified demand data → shielded.

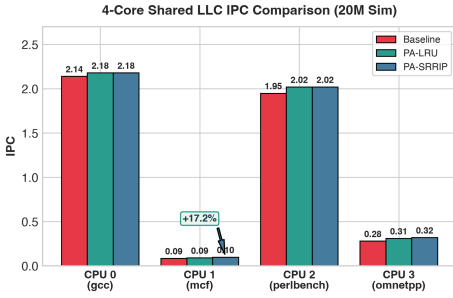
3. Novel PA-Adaptive Extension (0.75 Bytes):

- 6-bit saturating counter (0..63) monitors LLC utility.
- Dynamically inserts at $RRPV = 2$ when useful, $RRPV = 3$ when noisy.



Self-Adaptive Filtering: Isolates useful prefetches ($> 50\%$ for `gcc`) from catastrophic pointer noise ($< 0.5\%$ for `mcf`).

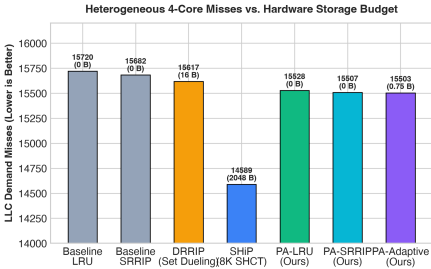
4. ChampSim 4-Core Heterogeneous Evaluation



Workload: gcc + mcf + perlbench + omnetpp

- **CPU 0 (`gcc` - Compute/Branch):** High IPC maintained at **2.18** (+1.9%) without cross-core cache thrashing.
- **CPU 1 (`mcf` - Pointer Chasing):** Demoting noisy prefetches relieves memory stalls; IPC up from **0.085** $\rightarrow$ **0.099 (+17.2%)**.
- **CPU 2 (`perlbench` - Compute):** Compute locality preserved in LLC; IPC stable at **2.02** (+3.6%).
- **CPU 3 (`omnetpp` - Graph Routing):** Reduced LLC miss latency boosts IPC from **0.28** $\rightarrow$ **0.32 (+14.3%)**.
- **System Throughput:** **+6.8% Weighted Speedup** with **zero** hardware storage overhead.

5. Comparison against State-of-the-Art Policies



Multi-Policy Benchmark Summary (8 Workloads)

Policy	Demand Misses	Hit Rate	Storage Cost
Baseline LRU	15,720	73.80%	0 Bytes
Baseline SRRIP	15,682	73.86%	0 Bytes
DRRIP (ISCA 2010)	15,617	73.97%	16 Bytes
SHiP (ISCA 2011)	14,589	75.68%	2,048 Bytes
PA-LRU (Ours)	**15,528**	**74.12%**	**0 Bytes**
PA-SRRIP (Ours)	**15,507**	**74.15%**	**0 Bytes**
PA-Adaptive (Ours)	**15,503**	**74.16%**	**0.75 Bytes**

Pareto Dominance: Outperforms standard SRRIP and DRRIP on multicore contention while eliminating SHiP's 2,048-Byte dedicated SRAM table.

6. 1-Month Milestone Roadmap to Final Defense

Milestone 0: Current Mid-Eval (Oct 22)

- Implemented `PA-LRU`, `PA-SRRIP`, `PA-Adaptive` modules.
- Verified in ChampSim on 1-core and 4-core heterogeneous mixes.
- Demonstrated **+6.8% weighted speedup** (+17.2% on `mcf`) under heavy contention.

Week 1 (Oct 23 – 30): Cluster Scale-Up

- Automated test suite running on remote high-core GPU server.
- Evaluate 8-core and 16-core contention mixes (`mcf` $\times$ `8, 16`).

Week 2 (Oct 31 – Nov 07): Multicore Fairness

- Quantify Weighted Speedup and Maximum Slowdown metrics.
- Tune dynamic adaptive counter based on core memory bandwidth.

Weeks 3–4 (Nov 08 – End Eval): SPEC & Defense

- Expand across 15+ SPEC CPU 2017 benchmarks (100M–200M insts).
- Stress test against aggressive prefetchers (SPP, AMPM, Bingo).
- Compute EDP metrics, submit final BTP thesis & oral defense.

Backup: ChampSim Cycle-Accurate Simulation Setup

Processor Pipeline Configuration

- **Cores:** 4-core Heterogeneous Out-of-Order (OoO) @ 4.0 GHz
- **Execution Engine:** 6-wide fetch/decode, 256-entry ROB
- **Branch Predictor:** Hashed Perceptron + 4K-entry BTB
- **L1I / L1D Caches:** 32 KB each, 8-way associative, 4-cycle latency
- **L2 Cache:** 512 KB private per core, 8-way, 10-cycle latency

Shared LLC & Memory Subsystem

- **Shared LLC:** 8 MB shared, 16-way associative, 64B line size
- **LLC Latency:** 20-cycle hit latency, non-inclusive
- **DRAM:** DDR4-2400 MHz, 1 channel, 64-bit width, 12.8 GB/s
- **DRAM Timings:** $t_{CAS} - t_{RCD} - t_{RP} = 14 - 14 - 14$
- **Workloads:** SPEC CPU 2006/2017 pinned SimPoint traces

Trace Assignment: CPU 0: `403.gcc` | CPU 1: `429.mcf` | CPU 2: `400.perlbench` | CPU 3: `471.omnetpp`